Implementation Decisions

The assessment brief and API contract intentionally leave several behaviors undefined. This document records the decisions made, so logic in the code can be reviewed against explicit, consistent rules rather than assumptions.

---

1. Threshold Boundaries (overlapping values)

All thresholds are treated as **half-open intervals**: `low ≤ x < next_level`. The upper bound of each range is exclusive; it belongs to the next category.

Level	Wind (mph)	Rain (in/hr)	Snow (in/hr)
Low	x < 25	x < 0.10	x < 0.5
Moderate	25 ≤ x < 35	0.10 ≤ x < 0.25	0.5 ≤ x < 1.0
High	35 ≤ x < 45	0.25 ≤ x < 0.50	1.0 ≤ x < 2.0
Severe	45 ≤ x < 55	0.50 ≤ x ≤ 1.00	2.0 ≤ x ≤ 3.0
No Travel	x ≥ 55	x > 1.00	x > 3.0

(Rain/Snow Severe upper bound is inclusive per the brief's stated "0.50–1.00" / "2.0–3.0" ranges; No Travel starts strictly above that.)

---

2. Combining Wind, Rain, and Snow Risk

A checkpoint's **base risk** = the **most severe** of the three individual classifications (`max(wind_level, rain_level, snow_level)`).

Severity order used for comparison: `low < moderate < high < severe < no_travel`

Load-override rules (Section 4.2 of requirements) are then applied **on top** of this base risk, and can only escalate it, never reduce it:

1. `wind ≥ 55 mph` → `no_travel` (regardless of load)
2. `45 ≤ wind ≤ 54` and `load > 30,000 lb` → `no_travel`
3. `35 ≤ wind ≤ 44` and `load > 40,000 lb` → at least `severe` (if base risk was already higher, e.g. `no_travel` from rain, it is kept)

---

3. Converting Checkpoint Risk into Route-Level Miles

Each checkpoint represents the **segment ending at that checkpoint**:

- Segment 1 = origin → checkpoint 1
- Segment N = checkpoint (N-1) → checkpoint N
- Final segment = last checkpoint → destination (inherits the last checkpoint's risk)

A segment's risk level = the risk level of its **ending checkpoint**. Segment length = `distance_from_origin_miles` difference from the previous point.

From this:

- `severe_miles` = sum of segment lengths where risk = `severe`
- `high_miles` = sum of segment lengths where risk = `high`
- `average_risk_score` = distance-weighted mean of numeric severity:

  `low = 0, moderate = 1, high = 2, severe = 3, no_travel = 4`

---

4. No Travel and Route Eligibility

- Any route containing **at least one** `no_travel` checkpoint is marked `"eligible": false` and is **excluded from ranking** (never recommended).
- If **all** returned routes are ineligible, `recommended_route_id: null`, and a top-level warning explains why (e.g. `"All routes contain a No Travel checkpoint; no safe route available."`).
- Ineligible routes are still returned in the `routes` array (so the UI can display them, greyed out / flagged) — they are just never recommended.

---

5. Fewer Than 3 Routes / Missing Weather Data

- **Routing provider returns < 3 routes:** return only what is available. Never fabricate a route. Add a top-level warning, e.g. `"Only 2 route option(s) returned by the routing provider."`
- **Weather unavailable at a checkpoint:** set `risk_level: "unknown"` (not `low`). Add a checkpoint-level warning, e.g. `"Weather data unavailable for this checkpoint."` A route containing `unknown` checkpoints is **not** auto-excluded, but a route-level warning is added noting the analysis is incomplete, and `unknown` checkpoints are excluded from `average_risk_score` and mile totals (not counted as any risk level).

---

6. Heatmap Reference Time & Forecast Hours

- `reference_time` = the backend server's timestamp at the moment the request is processed (`timezone.now()`), **not** the trip's departure time. This matches the weather provider's forecast baseline (forecasts are issued "from now", not "from departure").
- `available_forecast_hours` = `0` through `48` inclusive, at **hourly** resolution (matches the weather provider's native hourly granularity), i.e. `[0, 1, 2, ..., 48]`.
- The frontend slider maps directly to this array; selecting hour `H` shows forecast data for `reference_time + H hours` along the corridor.

Additional Implementation Notes (not explicitly asked, but needed)

- **Checkpoint ETA calculation:** interpolated from the routing provider's per-step distance/duration data (cumulative time at cumulative distance), not a flat average-speed assumption — more accurate on routes with mixed highway/surface-street segments.

- **Heatmap corridor points:** reuses checkpoint coordinates from all routes (not an independent grid), keeping the heatmap tied directly to roads actually being compared.
- **Persistence:** no database is used. The API is stateless — each request is fully analyzed and returned in one response, per the architecture note that persistence is not required by the brief.
- **Provider errors:** routing/geocoding/weather provider failures that block analysis return `502 Bad Gateway`; invalid request input returns `400 Bad Request`, using the contract's `{ "error": { "code", "message" } }` shape. Internal provider error details are logged server-side only.